

НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
“КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО”



Технології графічного процесінгу

Масштабування обчислень на кількох GPU

Практична робота #3

Виконав(ла):
Олександр Ковальов
Група:
ІМ-51мн
Курс:
1

19 квітня 2026 р.

1 Завдання

Використати датасет CIFAR-10 для побудови та навчання базової згорткової нейронної мережі у одно-GPU режимі з фіксацією ключових метрик, зокрема часу навчання, значення функції втрат і точності класифікації. Після цього слід реалізувати розподілене навчання тієї ж моделі з використанням підходу **data parallelism** (розподіл батчів між кількома GPU з подальшою агрегацією градієнтів), а також підходу **model parallelism** (розподіл шарів або частин моделі між кількома GPU), і виконати повторне навчання з аналогічною фіксацією метрик. Отримані результати необхідно порівняти, проаналізувавши вплив кожного підходу на час навчання, точність моделі та ефективність використання обчислювальних ресурсів, включаючи оцінку прискорення (**speedup**), ефективності масштабування та накладних витрат на міжпроцесорну взаємодію. На основі проведених експериментів потрібно сформулювати висновки щодо переваг і обмежень **data parallelism** та **model parallelism**, визначити сценарії їх доцільного застосування, а також пояснити причини можливих втрат продуктивності або відсутності очікуваного прискорення. Результати виконання роботи необхідно оформити у вигляді Jupyter Notebook, який містить програмну реалізацію, експериментальні дослідження та отримані результати, а також підготувати аналітичний звіт з обґрунтованими висновками щодо ефективності використаних підходів.

2 Аналіз методів паралельного навчання та результати експериментів

Метою даного розділу є детальне дослідження та порівняння ефективності різних стратегій розподілених обчислень при навчанні нейронних мереж. У ході практичної роботи було реалізовано та протестовано три основні сценарії: базове навчання на одному графічному процесорі, паралелізм даних та паралелізм моделі. Як об'єкт дослідження виступала згорткова нейронна мережа SimpleCNN, що класифікує зображення з набору даних CIFAR-10. Експерименти проводилися в середовищі Kaggle з використанням двох графічних прискорювачів NVIDIA Tesla T4. Для забезпечення об'єктивності порівняння параметри навчання, такі як розмір батчу (256), швидкість навчання та кількість епох, залишалися незмінними для кожного тесту.

Перший експеримент, проведений на одному GPU, продемонстрував базовий час виконання етапного завдання у 22.87 секунди. Цей показник став точкою відліку для розрахунку прискорення (Speedup). Точність моделі на третій епосі досягла приблизно 70 відсотків при значенні функції втрат 0.8530. Ці результати підтвердили адекватність обраної архітектури для вирішення поставленої задачі класифікації.

```
--- STARTING SINGLE GPU EXPERIMENT ---  
Epoch [1/3] - Loss: 1.5254, Accuracy: 44.30%  
Epoch [2/3] - Loss: 1.0624, Accuracy: 62.16%  
Epoch [3/3] - Loss: 0.8530, Accuracy: 69.89%  
Total Training Time: 22.87 seconds
```

Рис. 1: Перший експеримент.

Другий етап дослідження був присвячений стратегії Data Parallelism. При цьому підході модель SimpleCNN дублювалася на обох доступних GPU, а вхідний батч даних розділявся навпіл. Результати виявилися неочікуваними з точки зору чистої продуктивності: час навчання зріс до 25 секунд, що відповідає значенню Speedup близько 0.91. Таке сповільнення свідчить про те, що для невеликих моделей на кшталт SimpleCNN накладні витрати на передачу ваг моделі та синхронізацію градієнтів між картами через шину PCIe значно перевищують вигоду від паралельної обробки даних. Велика частина часу витрачалася на комунікаційні операції, а не на самі математичні обчислення.

```
--- STARTING DATA PARALLELISM EXPERIMENT ---  
Epoch [1/3] - Loss: 1.4932, Accuracy: 45.65%  
Epoch [2/3] - Loss: 1.0376, Accuracy: 62.87%  
Epoch [3/3] - Loss: 0.8173, Accuracy: 71.17%  
Total Training Time: 25.03 seconds
```

Рис. 2: Другий експеримент.

Третій експеримент фокусувався на підході Model Parallelism, де архітектура мережі була розділена за функціональною ознакою: згорткові шари виконувалися на першому пристрої, а повнозв'язні – на другому. Час навчання склав 21.73 секунди, що забезпечило мінімальне прискорення у 5 відсотків порівняно з базовим режимом ($\text{Speedup} = 1.05246203$). Хоча цей метод виявився дещо ефективнішим за розподіл даних у даних умовах, він все одно не показав лінійного прискорення. Головною причиною є послідовна залежність обчислень: другий GPU змушений простоювати, поки перший не завершить обробку початкових шарів і не передасть отримані активації через інтерфейс зв'язку.

```
--- STARTING MODEL PARALLELISM EXPERIMENT ---  
Epoch [1/3] - Loss: 1.4809, Accuracy: 46.14%  
Epoch [2/3] - Loss: 1.0329, Accuracy: 63.08%  
Epoch [3/3] - Loss: 0.8217, Accuracy: 71.03%  
Total Training Time: 21.73 seconds
```

Рис. 3: Третій експеримент.

2.1 Висновки

На основі проведених експериментальних досліджень можна стверджувати, що вибір стратегії паралелізму критично залежить від масштабу моделі та обсягу обчислень на один параметр. Результати практичної роботи наочно продемонстрували закон Амдала в дії: присутність непаралельних ділянок, таких як синхронізація градієнтів та послідовна передача даних між шарами, обмежує максимальне можливе прискорення системи. Для відносно простих архітектур, що працюють з малими вхідними даними, використання кількох GPU може не лише не дати очікуваного приросту швидкості, а й призвести до деградації продуктивності через високі накладні витрати на міжпроцесорну взаємодію.

Підхід Data Parallelism виявився найменш ефективним для поточної задачі, оскільки час на копіювання моделі та агрегацію градієнтів нівелював вигоду від розподілу батчу. Цей метод доцільно використовувати лише для важких моделей, де час однієї ітерації навчання значно перевищує час передачі даних. З іншого боку, Model Parallelism показав кращу стабільність, проте його ефективне застосування потребує впровадження конвеєризації (pipelining), щоб уникнути простою обчислювальних вузлів. Загалом, для навчання базових мереж на невеликих датасетах найбільш раціональним з точки зору ресурсів залишається використання одного потужного графічного процесора, оскільки складність налаштування розподіленої інфраструктури в таких випадках не виправдовується отриманим мізерним прискоренням.

3 Лістинг.

Лістинг 1: Код.

```

import torch
import torch.nn as nn
import torch.optim as optim
import torchvision
import torchvision.transforms as transforms
import time
import torch.nn.functional as F

# Check devices
num_gpus = torch.cuda.device_count()
print(f"Available_GPUs: {num_gpus}")
if num_gpus > 0:
    for i in range(num_gpus):
        print(f"GPU_{i}: {torch.cuda.get_device_name(i)}")

# Prepare dataset and dataloaders
transform = transforms.Compose([
    transforms.ToTensor(),
    transforms.Normalize((0.5, 0.5, 0.5), (0.5, 0.5, 0.5))
])

trainset = torchvision.datasets.CIFAR10(root='./data', train=True, download=True, transform=transform)
trainloader = torch.utils.data.DataLoader(trainset, batch_size=256, shuffle=True, num_workers=4)

testset = torchvision.datasets.CIFAR10(root='./data', train=False, download=True, transform=transform)
testloader = torch.utils.data.DataLoader(testset, batch_size=256, shuffle=False, num_workers=4)

# Base Model for Single GPU and Data Parallelism
class SimpleCNN(nn.Module):
    def __init__(self):
        super(SimpleCNN, self).__init__()
        self.conv1 = nn.Conv2d(3, 64, kernel_size=3, padding=1)
        self.conv2 = nn.Conv2d(64, 128, kernel_size=3, padding=1)
        self.conv3 = nn.Conv2d(128, 256, kernel_size=3, padding=1)
        self.pool = nn.MaxPool2d(2, 2)
        self.fc1 = nn.Linear(256 * 4 * 4, 512)
        self.fc2 = nn.Linear(512, 10)
        self.dropout = nn.Dropout(0.25)

    def forward(self, x):
        x = self.pool(F.relu(self.conv1(x)))
        x = self.pool(F.relu(self.conv2(x)))
        x = self.pool(F.relu(self.conv3(x)))
        x = x.view(-1, 256 * 4 * 4)
        x = self.dropout(F.relu(self.fc1(x)))
        x = self.fc2(x)
        return x

# Model specifically split for Model Parallelism
class SimpleCNN_MP(nn.Module):

```

```

def __init__(self):
    super(SimpleCNN_MP, self).__init__()
    self.dev0 = 'cuda:0'
    self.dev1 = 'cuda:1'

    # Put feature extraction on GPU 0
    self.features = nn.Sequential(
        nn.Conv2d(3, 64, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
        nn.Conv2d(64, 128, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2),
        nn.Conv2d(128, 256, kernel_size=3, padding=1), nn.ReLU(), nn.MaxPool2d(2, 2)
    ).to(self.dev0)

    # Put classification on GPU 1
    self.classifier = nn.Sequential(
        nn.Linear(256 * 4 * 4, 512), nn.ReLU(), nn.Dropout(0.25),
        nn.Linear(512, 10)
    ).to(self.dev1)

    def forward(self, x):
        x = x.to(self.dev0)
        x = self.features(x)
        x = x.view(-1, 256 * 4 * 4)
        x = x.to(self.dev1) # Transfer data to GPU 1
        x = self.classifier(x)
        return x

    def train_experiment(model, trainloader, criterion, optimizer, num_epochs=3, is_mp=
model.train()
start_time = time.time()

    for epoch in range(num_epochs):
        running_loss = 0.0
        correct = 0
        total = 0

        for inputs, labels in trainloader:
            if is_mp:
                # For Model Parallelism: inputs go to GPU 0, targets go to GPU 1 (where output is)
                inputs = inputs.to('cuda:0')
                labels = labels.to('cuda:1')
            else:
                # For Single / Data Parallelism: everything goes to GPU 0 (DataParallel handles the
                inputs = inputs.to('cuda:0')
                labels = labels.to('cuda:0')

            optimizer.zero_grad()
            outputs = model(inputs)
            loss = criterion(outputs, labels)
            loss.backward()
            optimizer.step()

            running_loss += loss.item()
            _, predicted = torch.max(outputs.data, 1)
            total += labels.size(0)

```

```
correct += (predicted == labels).sum().item()

epoch_loss = running_loss / len(trainloader)
epoch_acc = 100 * correct / total
print(f"Epoch_{epoch+1}/{num_epochs}]_Loss:_{epoch_loss:.4f},_Accuracy:_{epoch_acc:.4f}")

total_time = time.time() - start_time
print(f"Total_Training_Time:_{total_time:.2f}_seconds\n")
return total_time, epoch_loss, epoch_acc

print("——_STARTING_SINGLE_GPU_EXPERIMENT_——")
model_single = SimpleCNN().to('cuda:0')
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_single.parameters(), lr=0.001)

time_single, loss_single, acc_single = train_experiment(
    model_single, trainloader, criterion, optimizer, num_epochs=3, is_mp=False
)

print("——_STARTING_DATA_PARALLELISM_EXPERIMENT_——")
model_dp = SimpleCNN()
if torch.cuda.device_count() > 1:
    model_dp = nn.DataParallel(model_dp)
model_dp.to('cuda:0')

criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_dp.parameters(), lr=0.001)

time_dp, loss_dp, acc_dp = train_experiment(
    model_dp, trainloader, criterion, optimizer, num_epochs=3, is_mp=False
)

print("——_STARTING_MODEL_PARALLELISM_EXPERIMENT_——")
model_mp = SimpleCNN_MP()
criterion = nn.CrossEntropyLoss()
optimizer = optim.Adam(model_mp.parameters(), lr=0.001)

time_mp, loss_mp, acc_mp = train_experiment(
    model_mp, trainloader, criterion, optimizer, num_epochs=3, is_mp=True
)
```